BÀI TẬP GIỮA KỲ · KHOÁ CKA 2026

Triển khai ứng dụng Web CDB Kanban với Docker và Kubernetes (kubeadm) trên Azure

Báo cáo kiến trúc hạ tầng Cloud, tách tầng ứng dụng, đóng gói Docker đa phiên bản và vận hành High Availability trên cụm 3 node

MÃ NGUỒN & IMAGE REGISTRY

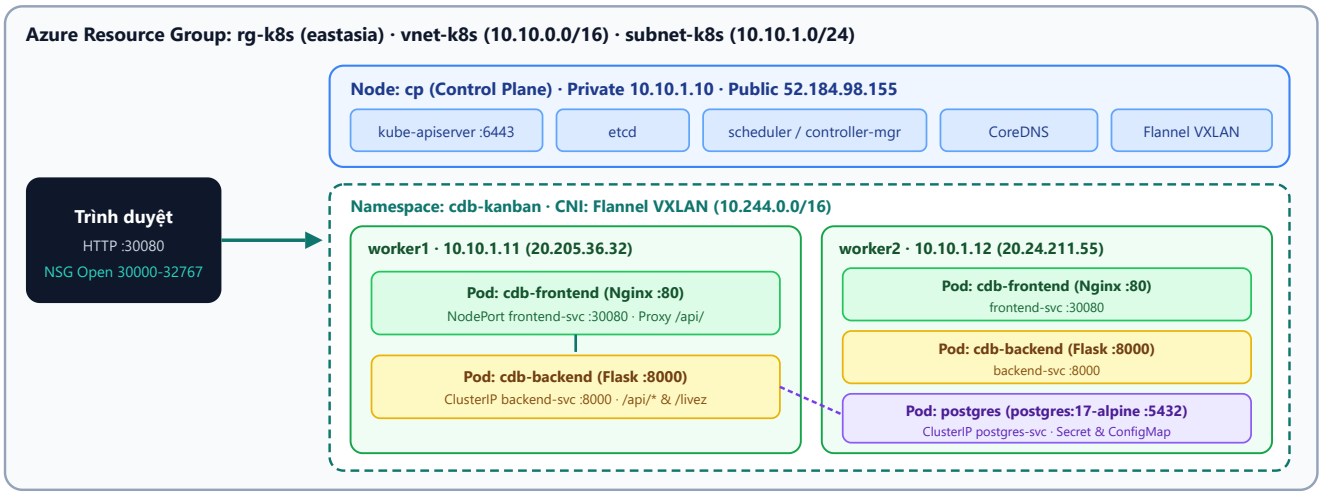
GitHub: `github.com/quanq026/cdb-kanban-lab_k8s`
Docker Hub: `quanq026/cdb-kanban-frontend`, `cdb-kanban-backend` (`v1` , `v2`)

NỀN TẢNG & ENDPOINT DEMO

Azure (`rg-k8s` , `eastasia`) · K8s `v1.35.10` · `containerd://2.4.2`
NodePort: `http://20.205.36.32:30080` · `20.24.211.55:30080`

1. Sơ đồ kiến trúc Cluster & Hạ tầng Microsoft Azure

Hệ thống được xây dựng mới hoàn toàn trong Resource Group `rg-k8s` (region `eastasia`) trên một mạng ảo riêng (`vnet-k8s` : `10.10.0.0/16` , `subnet-k8s` : `10.10.1.0/24`). Cổng truy cập từ ngoài vào kiểm soát bằng Network Security Group `nsg-k8s` .



Thành phần Azure	Tên tài nguyên	Thông số kỹ thuật
Mạng & Bảo mật	`vnet-k8s` , `subnet-k8s` , `nsg-k8s`	Dải `10.10.0.0/16` (subnet `10.10.1.0/24`); NSG mở TCP `22` , `80` , `443` , `30000-32767` (priority 1000–1030)
3 Máy ảo (VM)	`cp` , `worker1` , `worker2`	SKU `Standard_B2als_v2` (2 vCPU, 3.8 GiB RAM, `x86_64`), ổ OS `StandardSSD_LRS` , Ubuntu 24.04.5 LTS
IP Public tĩnh	Standard Static IPv4	`cp` : `52.184.98.155` · `worker1` : `20.205.36.32` · `worker2` : `20.24.211.55`

2. Tách tầng Web & Đóng gói Docker theo phiên bản

Từ ứng dụng Kanban nguyên khối ban đầu (`cdb-kanban-lab`), mã nguồn được tách vào repository mới `quanq026/cdb-kanban-lab_k8s` gồm ba thành phần độc lập:

- **Frontend (`frontend/` — `nginx:1.27-alpine` , ~73.7 MB):** Phục vụ các trang HTML/CSS/JS tĩnh (`/` , `/login` , `/register`), định tuyến toàn bộ lời gọi `/api/` tới `http://backend-svc:8000` , cung cấp endpoint `/healthz` và

chạy script `99-pod-info.sh` lúc khởi động để sinh `/frontend-info.json` hiển thị version + tên Pod lên giao diện.

- **Backend (`backend/` — `python:3.12-slim` + Flask + Gunicorn, ~217 MB):** Chỉ phục vụ REST API trên cổng `8000` (chạy user non-root `UID 10001`). Thêm biến môi trường `COOKIE_SECURE` (đặt `"false"` trong K8s cho truy cập HTTP NodePort), thêm `/livez` cho liveness probe và mở rộng `/api/health` trả thêm `version` và hostname `pod` .
- **Database (`db/schema.sql` — `postgres:17-alpine`):** Nạp schema bảng `users` , `tasks` , index `idx_tasks_user_status_position` và trigger `trg_tasks_updated_at` qua `ConfigMap` (`/docker-entrypoint-initdb.d/01-schema.sql`).

Cả hai `Dockerfile` nhận build-arg `ARG APP_VERSION=v1` (hoặc `v2`) và push lên Docker Hub với 4 tag tường minh, **tuyệt đối không dùng** `:latest` : `quanq026/cdb-kanban-frontend:v1` , `:v2` và `quanq026/cdb-kanban-backend:v1` , `:v2` .

3. Các bước cài đặt Kubernetes (`kubeadm`) & Triển khai YAML

1. **Chuẩn bị 3 node:** Tắt swap (`swapoff -a` , cập nhật `/etc/fstab`), cấu hình phân giải tên `cp` / `worker1` / `worker2` trong `/etc/hosts` , nạp module `overlay` và `br_netfilter` , bật `net.ipv4.ip_forward = 1` và `net.bridge.bridge-nf-call-iptables = 1` .
2. **Cấu hình Container Runtime:** Tái tạo `/etc/containerd/config.toml` bằng `containerd config default` trên cả 3 node để mở lại plugin CRI (`disabled_plugins = []`) và đổi `SystemdCgroup = true` cho `containerd v2.4.2` . Cài đặt và hold `kubelet` , `kubeadm` , `kubectl` phiên bản `v1.35.10` .
3. **Khởi tạo Control Plane & Join Worker:** Chạy `sudo kubeadm init --apiserver-advertise-address=10.10.1.10 --pod-network-cidr=10.244.0.0/16 --node-name=cp` , cấu hình `~/.kube/config` , cài Flannel VXLAN (`kube-flannel.yml`) và join `worker1` , `worker2` .
4. **Khai báo và áp dụng YAML trong `k8s/` (`kubectl apply -f ./k8s/`):**
   - `00-namespace.yaml` : Namespace `cdb-kanban` .
   - `01-postgres.yaml` : `Secret/cdb-secrets` , `ConfigMap/postgres-initdb` , `Deployment/postgres` (1 replica, probe `pg_isready`) và `ClusterIP Service/postgres-svc:5432` .
   - `02-backend.yaml` : `ConfigMap/backend-config` (`COOKIE_SECURE="false"` , `DB_HOST="postgres-svc"`), `Deployment/cdb-backend` (`replicas: 2` , `topologySpreadConstraints` , `maxSurge: 1, maxUnavailable: 0` , `tolerationSeconds: 30`) và `ClusterIP Service/backend-svc:8000` .
   - `03-frontend.yaml` : `Deployment/cdb-frontend` (`replicas: 2` , cùng cấu hình HA & RollingUpdate) và `NodePort Service/frontend-svc` (`80:30080`).

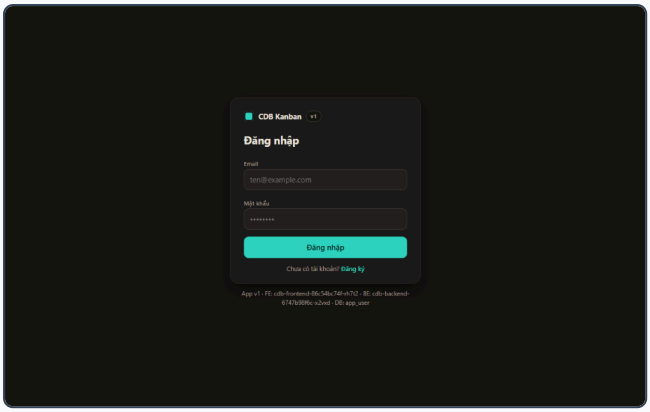
4. Ảnh chụp & Kết quả kiểm thử vận hành thực tế

4.1. Trạng thái Cluster 3 Node & Triển khai ban đầu (`v1`)

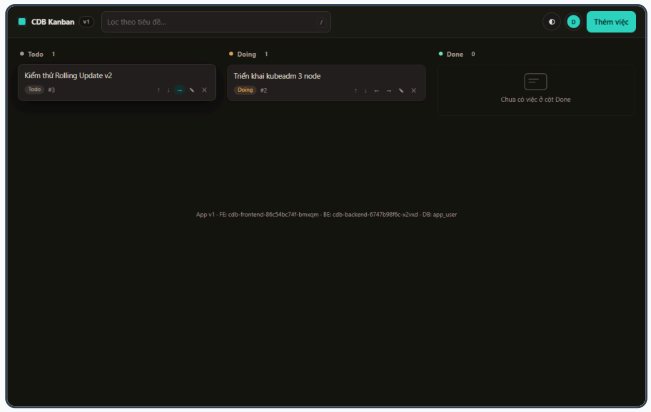
```
NAME      STATUS   ROLES           AGE   VERSION    INTERNAL-IP   OS-IMAGE             CONTAINER-RUNTIME
cp        Ready    control-plane   50m   v1.35.10   10.10.1.10    Ubuntu 24.04.5 LTS   containerd://2.4.2
worker1   Ready    <none>          50m   v1.35.10   10.10.1.11    Ubuntu 24.04.5 LTS   containerd://2.4.2
worker2   Ready    <none>          50m   v1.35.10   10.10.1.12    Ubuntu 24.04.5 LTS   containerd://2.4.2

NAME                                 READY   STATUS    RESTARTS   AGE   IP            NODE
pod/cdb-backend-6747b98f6c-bjctm     1/1     Running   0          60m   10.244.1.9    worker1
pod/cdb-backend-6747b98f6c-x2vxd     1/1     Running   0          59m   10.244.2.13   worker2
pod/cdb-frontend-86c54bc74f-bmxqm    1/1     Running   0          60m   10.244.1.10   worker1
pod/cdb-frontend-86c54bc74f-rh7t2    1/1     Running   0          59m   10.244.2.12   worker2
pod/postgres-b457ffcd6-9srxc         1/1     Running   0          64m   10.244.2.2    worker2

NAME                   TYPE        CLUSTER-IP       PORT(S)        SELECTOR
service/backend-svc    ClusterIP   10.99.248.222    8000/TCP       app=cdb-backend
service/frontend-svc   NodePort    10.106.144.93    80:30080/TCP   app=cdb-frontend
service/postgres-svc   ClusterIP   10.107.241.226   5432/TCP       app=postgres
```



Hình 1: Truy cập thực tế `http://20.24.211.55:30080/login` hiển thị badge `v1` và tên Pod FE/BE



Hình 2: Tạo task và chuyển cột `Todo → Doing` trên `http://20.205.36.32:30080` (lưu vào PostgreSQL)

4.2. Diễn tập Scale 4 Replicas, Rolling Update (v1 → v2) & Rollback

```
$ kubectl scale deployment cdb-frontend -n cdb-kanban --replicas=4
deployment.apps/cdb-frontend scaled
$ kubectl get pods -n cdb-kanban -l app=cdb-frontend -o wide
NAME                            READY   STATUS    RESTARTS   AGE   IP            NODE
cdb-frontend-7c4fb64f7c-d4vll   1/1     Running   0          8s    10.244.1.4    worker1
cdb-frontend-7c4fb64f7c-lzq8k   1/1     Running   0          72s   10.244.1.3    worker1
cdb-frontend-7c4fb64f7c-s4xhv   1/1     Running   0          8s    10.244.2.5    worker2
cdb-frontend-7c4fb64f7c-xxmb6   1/1     Running   0          72s   10.244.2.4    worker2
```

```
$ kubectl set image deployment/cdb-backend backend=quanq026/cdb-kanban-backend:v2 -n cdb-kanban
$ kubectl set image deployment/cdb-frontend frontend=quanq026/cdb-kanban-frontend:v2 -n cdb-kanban
$ curl -s http://20.205.36.32:30080/api/health
{"db_user":"app_user","ok":true,"pod":"cdb-backend-5dfcf77654-2bwk7","status":"ok","version":"v2"}

$ kubectl rollout undo deployment/cdb-backend -n cdb-kanban
$ kubectl rollout undo deployment/cdb-frontend -n cdb-kanban
$ curl -s http://20.205.36.32:30080/api/health
{"db_user":"app_user","ok":true,"pod":"cdb-backend-784bfd57c5-tdqtp","status":"ok","version":"v1"}
```

Khi cập nhật lên `:v2`, Kubernetes tạo Pod mới đạt `readinessProbe` trước khi tắt Pod cũ (`maxUnavailable: 0`); giao diện chuyển badge sang `v2` và toàn bộ user/task đã tạo ở phiên bản `v1` trong PostgreSQL vẫn giữ nguyên. Sau khi chạy `kubectl rollout undo`, API `/api/health` xác nhận khôi phục về `version: "v1"`.

4.3. Thử nghiệm tắt 1 Worker Node (`worker1`) & Khôi phục

```
$ az vm deallocate -g rg-k8s -n worker1
$ kubectl get nodes
NAME      STATUS     ROLES           AGE     VERSION    INTERNAL-IP
cp        Ready      control-plane   9m28s   v1.35.10   10.10.1.10
worker1   NotReady   <none>          9m17s   v1.35.10   10.10.1.11
worker2   Ready      <none>          9m17s   v1.35.10   10.10.1.12

$ kubectl get pods -n cdb-kanban -o wide
cdb-backend-784bfd57c5-cflzp    1/1     Running       0   11s     10.244.2.10   worker2
cdb-backend-784bfd57c5-l79gs    1/1     Running       0   11s     10.244.2.11   worker2
cdb-backend-784bfd57c5-nwx2l    1/1     Terminating   0   99s     10.244.1.8    worker1
cdb-frontend-7c4fb64f7c-5x5hl   1/1     Running       0   106s    10.244.2.8    worker2
cdb-frontend-7c4fb64f7c-wlr7z   1/1     Running       0   99s     10.244.2.9    worker2
postgres-b457ffcd6-9srxc        1/1     Running       0   3m37s   10.244.2.2    worker2
```

Trong thử nghiệm, Pod `postgres` chạy trên `worker2` nên nhóm tiến hành tắt `worker1` bằng `az vm deallocate`. Node `worker1` chuyển sang `NotReady`; nhờ cấu hình `tolerationSeconds: 30` và `whenUnsatisfiable: ScheduleAnyway`, Kubernetes tự động khởi tạo Pod thay thế ngay trên `worker2` để đưa cả Backend lẫn Frontend về lại `2/2 Running`. Truy cập web qua IP node sống `http://20.24.211.55:30080` vẫn đăng nhập và thao tác Kanban bình thường. Sau khi bật lại `worker1` (`az vm start`), node tự động quay lại trạng thái `Ready`.

5. Khó khăn gặp phải & Giải pháp xử lý

1. Giới hạn SKU máy ảo & Quota IP Public trên Azure: Vùng `japaneast` chặn cấp phát VM B-series x64 và subscription giới hạn tối đa 3 Public IPv4 Standard mỗi region. *Giải pháp:* Chuyển sang vùng `eastasia` với dòng `Standard_B2als_v2` (tổng 6 vCPU, 3 IP Public Standard) vừa đủ cho mô hình 1 Control Plane + 2 Worker.

2. Mặc định `containerd.io` tắt CRI & sai Cgroup driver: Gói `containerd.io` cài từ Docker repo đặt sẵn `disabled_plugins = ["cri"]` và `SystemdCgroup = false` khiến `kubeadm init` lỗi. *Giải pháp:* Sinh lại `/etc/containerd/config.toml` từ `containerd config default` và chuyển `SystemdCgroup = true` trên cả 3 node.

3. Mạng Azure SDN chặn gói tin IP-in-IP: Giao tiếp Pod giữa hai worker dễ bị rớt nếu dùng overlay IP-in-IP. *Giải pháp:* Triển khai **Flannel VXLAN** (`10.244.0.0/16`) đóng gói UDP tương thích hoàn toàn với Azure VNet.

4. Trình duyệt không lưu Cookie phiên khi chạy qua HTTP NodePort: Bản Web ban đầu khóa cứng `SESSION_COOKIE_SECURE=True` (yêu cầu HTTPS). *Giải pháp:* Tách thành biến cấu hình `COOKIE_SECURE="false"` trong `ConfigMap/backend-config` để phiên đăng nhập hoạt động bình thường trên `http://<node-ip>:30080` .

5. Cân bằng Pod & Tránh kẹt `Pending` khi mất 1 Worker: Nếu dùng `whenUnsatisfiable: DoNotSchedule` khi tắt một worker, Pod thứ hai sẽ bị kẹt `Pending` . *Giải pháp:* Dùng `ScheduleAnyway` trong `topologySpreadConstraints` để vừa rải đều 1+1 khi đủ node, vừa dồn đủ 2/2 sang node sống khi có sự cố; kèm `tolerationSeconds: 30` để demo self-healing nhanh thay vì chờ 300s mặc định.

6. Tổng kết & Đối chiếu tiêu chí hoàn thành

Hạng mục yêu cầu	Trạng thái triển khai & Kiểm chứng thực tế
Hạ tầng & Cluster K8s	3 VM Azure (`cp` , `worker1` , `worker2`) cài `kubeadm v1.35.10` + Flannel VXLAN; cả 3 node ở trạng thái `Ready` .
Tách tầng & Docker Hub	Tách riêng `frontend/` (Nginx) & `backend/` (Flask/Gunicorn); build và push 4 image tag `:v1` , `:v2` (không dùng `:latest`).
Kubernetes Manifests	Khai báo đầy đủ trong `k8s/` (`Namespace` , `Secret` , `ConfigMap` , `Deployment` có probe, `ClusterIP` , `NodePort :30080`).
Vận hành HA & Tự phục hồi	Đã diễn tập thành công `scale` 2→4→2, rolling update `:v1→:v2` , `rollout undo` bảo toàn dữ liệu PostgreSQL và failover khi tắt `worker1` .
Giao diện & Nghiệm thu	Truy cập thực tế qua `http://20.205.36.32:30080` và `http://20.24.211.55:30080` , đăng ký/đăng nhập và thao tác Kanban Board ổn định.